Utilisation du Catalogue Tekton

Temps estimé nécessaire : 30 minutes

Bienvenue dans le laboratoire pratique pour l'utilisation du Catalogue Tekton. La communauté Tekton propose une large sélection de tâches et de pipelines réutilisables que vous pouvez utiliser dans vos workflows CI/CD, afin de ne pas avoir à tout écrire à partir de zéro. De nombreuses tâches couramment utilisées sont disponibles dans le Catalogue Tekton, qui est hébergé sur GitHub et indexé sur Artifact Hub pour la découverte.

Dans ce laboratoire, vous allez rechercher et utiliser une tâche maintenue par la communauté à partir du Catalogue Tekton.

Objectifs d'apprentissage

Après avoir terminé ce laboratoire, vous serez capable de :

- Utiliser le Catalogue Tekton pour installer la tâche git-clone
- Décrire les paramètres nécessaires pour utiliser la tâche git-clone
- Utiliser la tâche git-clone dans un pipeline Tekton pour cloner un dépôt Git

---

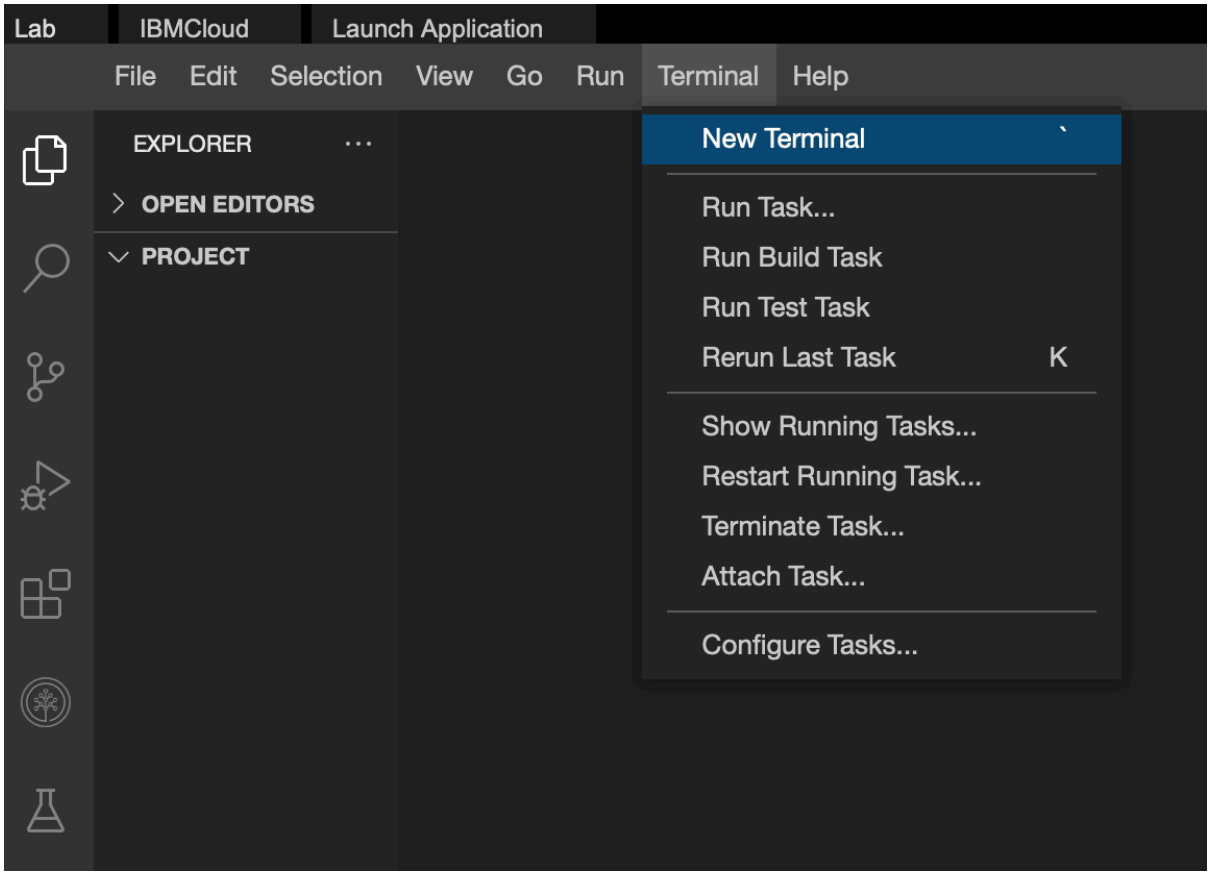
The content of this lab is licensed under Apache 2.0

Configurer l'environnement de laboratoire

Avant de commencer le laboratoire, complétez les étapes de configuration suivantes.

Ouvrir un terminal

Ouvrez une fenêtre de terminal en utilisant le menu de l'éditeur : Terminal > Nouveau terminal.



Dans le terminal, si vous n'êtes pas déjà dans le dossier `/home/project`, changez maintenant pour votre dossier de projet.

bash

```bash
cd /home/project
```

Run

The content of this lab is licensed under Apache 2.0

Cloner le dépôt de code

Maintenant, obtenez le code dont vous avez besoin pour tester. Pour ce faire, utilisez la commande `git clone` pour cloner le dépôt Git :

```bash
git clone https://github.com/ibm-developer-skills-network/wtecc-CICD_Pract
```

Run

Votre sortie devrait ressembler à l'image ci-dessous :

```
theia@theiaopenshift-rofrano:/home/project$ git clone https://github.com/ibm-developer-skills-network/wtecc-CICD_PracticeCode.git
Cloning into 'wtecc-CICD_PracticeCode'...
remote: Enumerating objects: 37, done.
remote: Counting objects: 100% (7/7), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 37 (delta 1), reused 4 (delta 0), pack-reused 30
Unpacking objects: 100% (37/37), done.
theia@theiaopenshift-rofrano:/home/project$
```

Changez de répertoire vers les Labs

Une fois que vous avez cloné le dépôt, changez de répertoire vers le dossier labs.

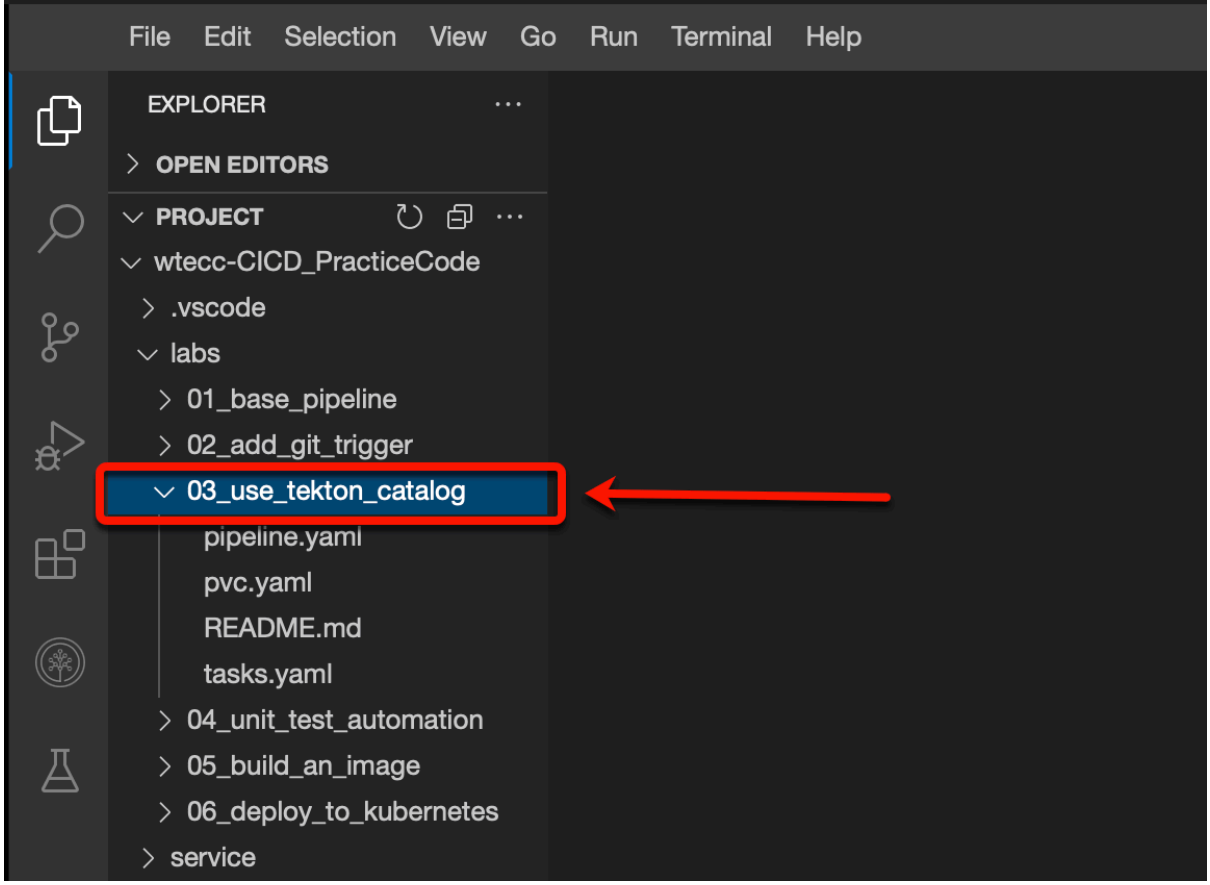
```bash
cd wtecc-CICD_PracticeCode/labs/03_use_tekton_catalog/
```

Run

Accédez au dossier Labs

Accédez au dossier `labs/03_use_tekton_catalog` dans le panneau d'exploration de gauche. Tout votre travail se fera avec les fichiers de ce dossier.

The content of this lab is licensed under Apache 2.0



Vous êtes maintenant prêt à commencer avec les prérequis dans la section suivante.

Optionnel

Si travailler dans le terminal devient difficile en raison de la longueur de l'invite de commande, vous pouvez la raccourcir en utilisant la commande suivante :

bash

```bash
export PS1="[\[\033[01;32m\]\u\[\033[00m\]: \[\033[01;34m\]\W\[\033[00m\]]
```

Run

The content of this lab is licensed under Apache 2.0

Prérequis

Ce laboratoire nécessite l'installation des tâches introduites dans les laboratoires précédents. Pour être sûr, appliquez les tâches précédentes à votre cluster avant de continuer :

```bash
kubectl apply -f tasks.yaml
```

Vous devriez voir une sortie similaire à ceci :

> ***Remarque : Si les tâches sont déjà installées, la sortie indiquera "configuré" au lieu de "créé."***

```text
task.tekton.dev/echo created
task.tekton.dev/checkout created
```

Vous êtes maintenant prêt à commencer le laboratoire.

The content of this lab is licensed under Apache 2.0

Étape 1 : Ajouter la tâche git-clone

Dans cette étape, vous remplacerez la tâche de checkout personnalisée créée précédemment par la tâche `git-clone` maintenue par la communauté. Bien que la tâche personnalisée ait été utile à des fins d'apprentissage, la tâche fournie par la communauté est plus robuste et mieux adaptée aux pipelines du monde réel.

La tâche `git-clone` fait partie du **Tekton Catalog** et est indexée sur **Artifact Hub** pour la découverte. Vous pouvez installer la tâche en utilisant l'une des options suivantes.

Option 1 : Découvrir via Artifact Hub et appliquer le manifeste brut

bash

```bash
kubectl apply -f https://github.com/tektoncd/catalog/raw/main/task/git-clo
```

Run

Option 2 : Installer directement depuis le catalogue Tekton

Artifact Hub indexe les tâches du catalogue Tekton officiel hébergé sur GitHub. Vous pouvez installer la même tâche directement depuis le dépôt source :

bash

```bash
kubectl apply -f https://raw.githubusercontent.com/tektoncd/catalog/main/t
```

Run

Les deux options installent la tâche git-clone dans votre cluster sous l'espace de noms actuellement actif.

Vérifiez l'installation

Après l'installation, confirmez que la tâche est disponible :

bash

The content of this lab is licensed under Apache 2.0

Vous devriez voir la tâche git-clone répertoriée dans la sortie.

The content of this lab is licensed under Apache 2.0

Étape 2 : Créer un espace de travail

En consultant les exigences de la tâche `git-clone`, vous constatez que bien qu'elle prenne en charge de nombreux paramètres supplémentaires par rapport à votre tâche `checkout` d'origine, elle ne *requiert* que deux choses :

1. L'URL d'un dépôt Git à cloner, fournie avec le paramètre `url`
2. Un espace de travail appelé `output`

Vous commencez par créer un `PersistentVolumeClaim` (PVC) à utiliser comme espace de travail :

Un espace de travail est un volume de disque qui peut être partagé entre les tâches. La manière de se lier aux volumes dans Kubernetes est avec un `PersistentVolumeClaim`.

Puisque la création de PVC est au-delà du cadre de ce laboratoire, un fichier `pvc.yaml` vous a été fourni avec ces contenus :

yaml

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pipelinerun-pvc
spec:
  storageClassName: skills-network-learner
  resources:
    requests:
      storage:  1Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
```

Appliquez la nouvelle définition de tâche au cluster :

bash

```bash
kubectl apply -f pvc.yaml
```

The content of this lab is licensed under Apache 2.0

Vous devriez voir la sortie suivante :

```text
persistentvolumeclaim/pipelinerun-pvc created
```

Vous pouvez maintenant référencer ce volume persistant par son nom `pipelinerun-pvc` lors de la création d'espaces de travail pour vos tâches Tekton.

The content of this lab is licensed under Apache 2.0

Étape 3 : Ajouter un Espace de Travail au Pipeline

Dans cette étape, vous ajouterez un espace de travail au pipeline en utilisant la revendication de volume persistant que vous venez de créer. Pour ce faire, vous allez modifier le fichier `pipeline.yaml` et ajouter une définition `workspaces:` comme première ligne sous `spec:` mais avant `params:` et l'appeler `pipeline-workspace`. Ensuite, vous ajouterez l'espace de travail à la tâche `clone` du pipeline et changerez la tâche pour référencer `git-clone` au lieu de votre tâche `checkout`.

Open pipeline.yaml in IDE

Votre Tâche

1. Modifiez le fichier `pipeline.yaml` et ajoutez une définition `workspaces:` comme première ligne sous `spec:` mais avant `params:` et appelez-la `pipeline-workspace`.

2. Ensuite, ajoutez l'espace de travail à la tâche `clone` après `name:` et appelez-le `output` car c'est le nom de l'espace de travail que la tâche `git-clone` recherchera.

3. Changez le nom de la référence de tâche (`taskRef`) dans la tâche `clone` pour référencer la tâche `git-clone` au lieu de `checkout`.

4. Enfin, changez le nom du paramètre `repo-url` en `url` car c'est le nom que la tâche `git-clone` attend, mais gardez le mappage de `$(params.repo-url)`, qui est ce que le pipeline attend. Renommez également le paramètre `branch` en `revision`, qui est ce que `git-clone` attend.

Indice

▶ **Cliquez ici pour un indice.**

Vérifiez que votre travail correspond à la solution ci-dessous.

The content of this lab is licensed under Apache 2.0

Solution

▶ **Cliquez ici pour la réponse.**

Appliquez le pipeline à votre cluster :

bash

```bash
kubectl apply -f pipeline.yaml
```

Run

Vous devriez voir une sortie similaire à ceci :

> ***Remarque : Si le pipeline original a déjà été créé, vous verrez le mot "configuré" au lieu de "créé."***

text

```text
$ kubectl apply -f pipeline.yaml
pipeline.tekton.dev/cd-pipeline created
```

Vous êtes maintenant prêt à exécuter votre pipeline.

The content of this lab is licensed under Apache 2.0

Étape 4 : Exécuter le Pipeline

Vous pouvez maintenant utiliser le CLI Tekton (`tkn`) pour créer un PipelineRun afin d'exécuter le pipeline.

Utilisez la commande suivante pour exécuter le pipeline, en passant l'URL du dépôt, la branche à cloner, le nom de l'espace de travail et le nom de la demande de volume persistant.

bash

```bash
tkn pipeline start cd-pipeline \
    -p repo-url="https://github.com/ibm-developer-skills-network/wtecc-CIC
    -p branch="main" \
    -w name=pipeline-workspace,claimName=pipelinerun-pvc \
    --showlog
```

Run

Vous devriez voir une sortie similaire à ceci :

text

```text
$ tkn pipeline start cd-pipeline \
>     -p repo-url="https://github.com/ibm-developer-skills-network/wtecc-C
>     -p branch="main" \
>     -w name=pipeline-workspace,claimName=pipelinerun-pvc \
>     --showlog
PipelineRun started: cd-pipeline-run-mndgw
Waiting for logs to be available...
```

Finalement, vous devriez voir la sortie des journaux.

> *Remarque : Il y aura plusieurs lignes de sortie de [clone: clone]. Celles-ci ne sont pas représentées ci-dessous pour plus de clarté.*

The content of this lab is licensed under Apache 2.0

```text
[clone : clone] <- There will be many lines from git-clone
[clone : clone] ...
[lint : echo-message] Calling Flake8 linter...
[tests : echo-message] Running unit tests with PyUnit...
[build : echo-message] Building image for https://github.com/ibm-developer
[deploy : echo-message] Deploying main branch of https://github.com/ibm-de
```

Vous pouvez toujours voir l'état d'exécution du pipeline en listant les PipelineRuns avec :

```bash
tkn pipelinerun ls
```

Run

Vous devriez voir :

```plaintext
NAME                    STARTED          DURATION     STATUS
cd-pipeline-run-mrg6g   45 seconds ago   18 seconds   Succeeded
```

Vous pouvez vérifier les journaux de la dernière exécution avec :

```bash
tkn pipelinerun logs --last
```

Run

The content of this lab is licensed under Apache 2.0

Conclusion

Félicitations ! Vous venez d'ajouter une tâche du catalogue Tekton au lieu de l'écrire vous-même. Vous devriez prendre l'habitude de toujours vérifier le catalogue Tekton avant d'écrire une tâche. Rappelez-vous : *"Une ligne de code que vous n'avez pas écrite est une ligne de code que vous n'avez pas à maintenir !"*

Dans ce laboratoire, vous avez appris à utiliser la tâche `git-clone` du catalogue Tekton. Vous avez appris à installer la tâche en appliquant le YAML du catalogue à votre cluster en utilisant `kubectl` et comment modifier votre pipeline pour référencer la tâche et configurer ses paramètres. Vous avez également appris à démarrer un pipeline avec la commande Tekton CLI `pipeline start` et à surveiller sa sortie en utilisant `--showlog`.

Prochaines Étapes

Dans le prochain laboratoire, vous utiliserez une combinaison de tâches écrites par vous-même et de tâches du catalogue pour compléter votre pipeline dans les laboratoires futurs. En attendant, essayez de configurer un pipeline pour créer une image avec Tekton à partir de l'un de vos propres dépôts de code.

Si vous êtes intéressé à continuer à apprendre sur Kubernetes et les conteneurs, vous pouvez obtenir votre propre cluster Kubernetes gratuit et votre propre IBM Container Registry gratuit.

Author(s)

Tapas Mandal
John J. Rofrano

The content of this lab is licensed under Apache 2.0

Étape 2 : Créer un espace de travail

En consultant les exigences de la tâche `git-clone`, vous constatez que bien qu'elle prenne en charge de nombreux paramètres supplémentaires par rapport à votre tâche `checkout` d'origine, elle ne *requiert* que deux choses :

1. L'URL d'un dépôt Git à cloner, fournie avec le paramètre `url`
2. Un espace de travail appelé `output`

Vous commencez par créer un `PersistentVolumeClaim` (PVC) à utiliser comme espace de travail :

Un espace de travail est un volume de disque qui peut être partagé entre les tâches. La manière de se lier aux volumes dans Kubernetes est avec un `PersistentVolumeClaim`.

Puisque la création de PVC est au-delà du cadre de ce laboratoire, un fichier `pvc.yaml` vous a été fourni avec ces contenus :

yaml

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pipelinerun-pvc
spec:
  storageClassName: skills-network-learner
  resources:
    requests:
      storage:  1Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
```

Appliquez la nouvelle définition de tâche au cluster :

bash

```bash
kubectl apply -f pvc.yaml
```

The content of this lab is licensed under Apache 2.0

Vous devriez voir la sortie suivante :

```text
persistentvolumeclaim/pipelinerun-pvc created
```

Vous pouvez maintenant référencer ce volume persistant par son nom `pipelinerun-pvc` lors de la création d'espaces de travail pour vos tâches Tekton.

← Étape 1 : Ajouter la tâche …

Étape 3 : Ajouter un Espa… →

The content of this lab is licensed under Apache 2.0